Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 26

Construct a C program to implement the file management operations.

Executable C code :

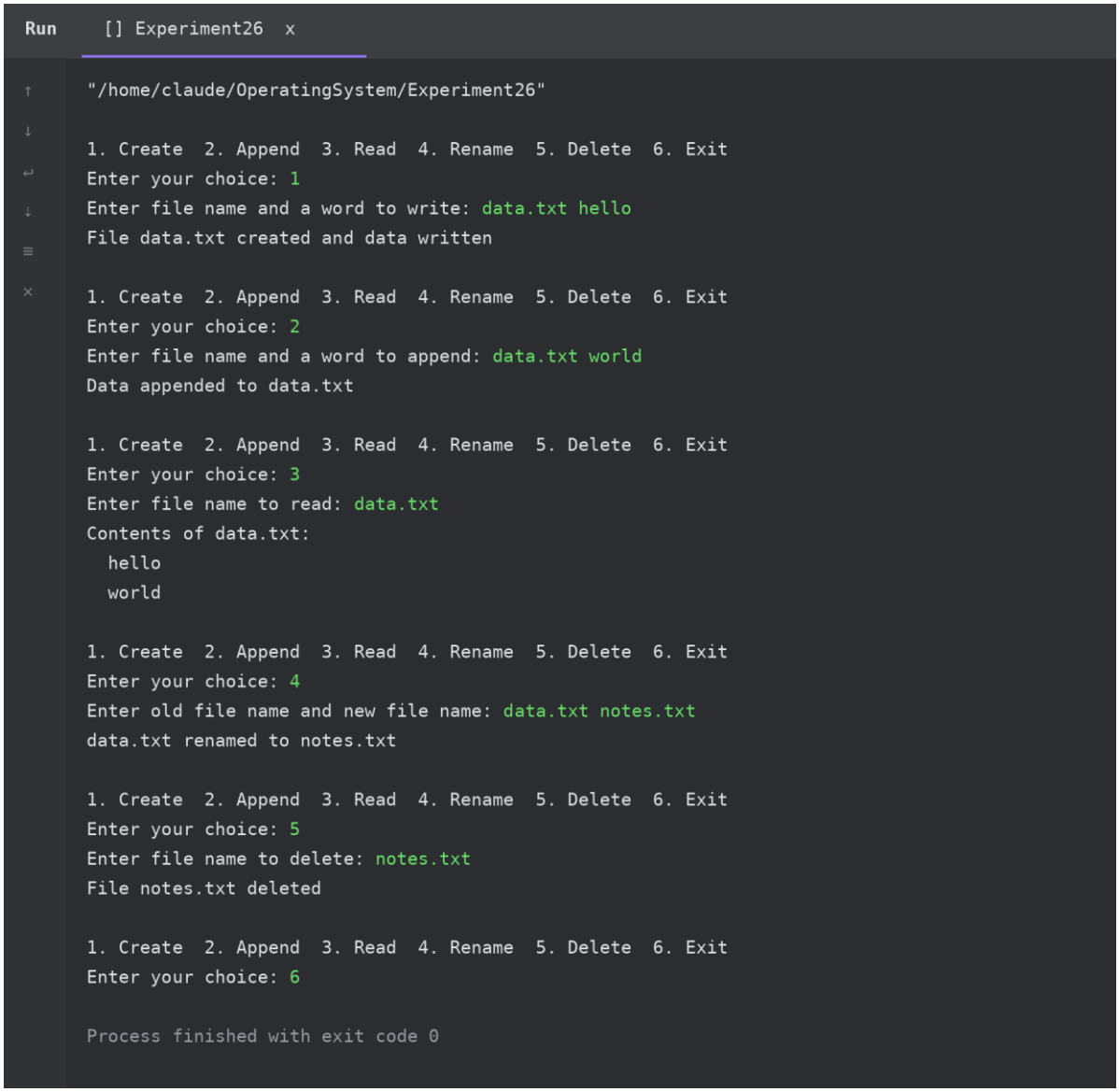
```
#include <stdio.h>

int main() {
    FILE *fp;
    char fname[50], newname[50], text[100], line[100];
    int choice;

    do {
        printf("\n1. Create  2. Append  3. Read  4. Rename  5. Delete  6.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter file name and a word to write: ");
            scanf("%s %s", fname, text);
            fp = fopen(fname, "w");
            if (fp == NULL) { printf("Cannot create file\n"); break; }
            fprintf(fp, "%s\n", text);
            fclose(fp);
            printf("File %s created and data written\n", fname);
            break;
        case 2:
            printf("Enter file name and a word to append: ");
            scanf("%s %s", fname, text);
            fp = fopen(fname, "a");
            if (fp == NULL) { printf("Cannot open file\n"); break; }
            fprintf(fp, "%s\n", text);
            fclose(fp);
            printf("Data appended to %s\n", fname);
            break;
        case 3:
            printf("Enter file name to read: ");
            scanf("%s", fname);
            fp = fopen(fname, "r");
            if (fp == NULL) { printf("File %s not found\n", fname); break; }
            printf("Contents of %s:\n", fname);
            while (fgets(line, sizeof(line), fp) != NULL) printf("  %s", line);
            fclose(fp);
            break;
        case 4:
            printf("Enter old file name and new file name: ");
            scanf("%s %s", fname, newname);
            if (rename(fname, newname) == 0) printf("%s renamed to %s\n", fname,
newname);
            else printf("Rename failed\n");
            break;
        case 5:
            printf("Enter file name to delete: ");
            scanf("%s", fname);
            if (remove(fname) == 0) printf("File %s deleted\n", fname);
            else printf("Delete failed\n");
            break;
        case 6:
            break;
```

```
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 6);
    return 0;
}
```

Output Screenshot:



Experiment 27

Develop a C program for simulating the function of ls UNIX Command.

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

int cmp(const void *a, const void *b) {
    return strcmp((const char *)a, (const char *)b);
}

int main() {
    char dirname[100], path[600], names[50][256], mode[11];
    int option, count = 0, i;
    DIR *dp;
    struct dirent *ent;
    struct stat st;

    printf("Enter directory name: ");
    scanf("%s", dirname);
    printf("Enter 1 for ls, 2 for ls -l: ");
    scanf("%d", &option);

    dp = opendir(dirname);
    if (dp == NULL) {
        printf("Cannot open directory %s\n", dirname);
        return 1;
    }
    while ((ent = readdir(dp)) != NULL) {
        if (ent->d_name[0] != '.') strcpy(names[count++], ent->d_name);   // hide
dot files like ls
    }
    closedir(dp);
    qsort(names, count, 256, cmp);                                          // ls
lists in sorted order

    printf("\n");
    for (i = 0; i < count; i++) {
        if (option == 1) {
            printf("%s  ", names[i]);
        } else {
            sprintf(path, "%s/%s", dirname, names[i]);
            stat(path, &st);
            strcpy(mode, "----------");
            if (S_ISDIR(st.st_mode)) mode[0] = 'd';
            if (st.st_mode & S_IRUSR) mode[1] = 'r';
            if (st.st_mode & S_IWUSR) mode[2] = 'w';
            if (st.st_mode & S_IXUSR) mode[3] = 'x';
            if (st.st_mode & S_IRGRP) mode[4] = 'r';
            if (st.st_mode & S_IWGRP) mode[5] = 'w';
            if (st.st_mode & S_IXGRP) mode[6] = 'x';
            if (st.st_mode & S_IROTH) mode[7] = 'r';
            if (st.st_mode & S_IWOTH) mode[8] = 'w';
            if (st.st_mode & S_IXOTH) mode[9] = 'x';
            printf("%s %6ld %s\n", mode, (long)st.st_size, names[i]);
        }
    }
    if (option == 1) printf("\n");
    return 0;
}
```

Output Screenshot:



Experiment 28

Write a C program for simulation of GREP UNIX command

Executable C code :

```
#include <stdio.h>
#include <string.h>

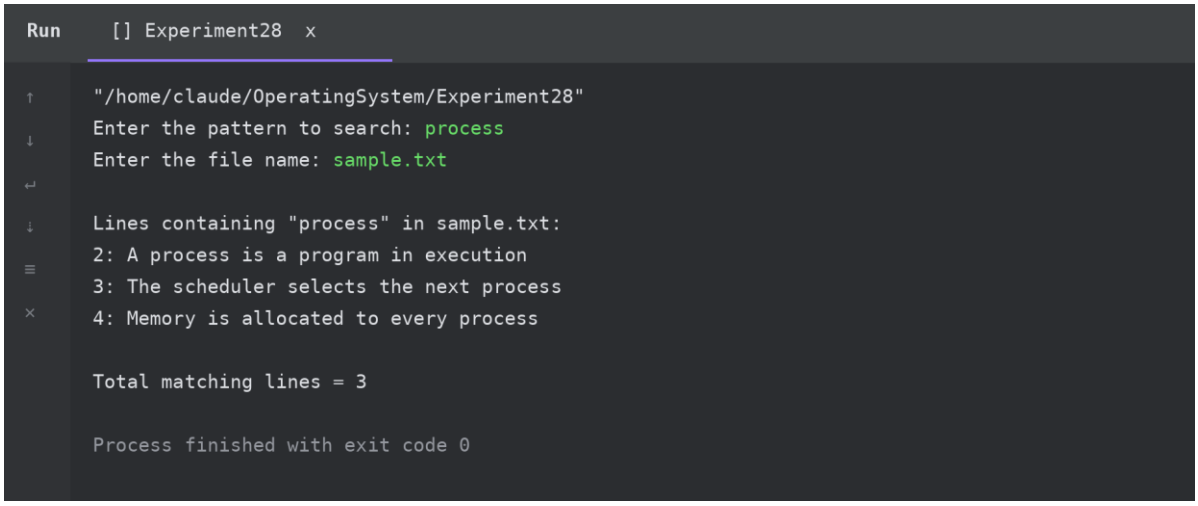
int main() {
    FILE *fp;
    char pattern[50], fname[50], line[200];
    int lineno = 0, count = 0;

    printf("Enter the pattern to search: ");
    scanf("%s", pattern);
    printf("Enter the file name: ");
    scanf("%s", fname);

    fp = fopen(fname, "r");
    if (fp == NULL) {
        printf("Cannot open file %s\n", fname);
        return 1;
    }

    printf("\nLines containing \"%s\" in %s:\n", pattern, fname);
    while (fgets(line, sizeof(line), fp) != NULL) {
        lineno++;
        if (strstr(line, pattern) != NULL) {     // pattern found in this line
            printf("%d: %s", lineno, line);
            count++;
        }
    }
    printf("\nTotal matching lines = %d\n", count);
    fclose(fp);
    return 0;
}
```

Output Screenshot:



Experiment 29

Write a C program to simulate the solution of Classical Process Synchronization Problem

Executable C code :

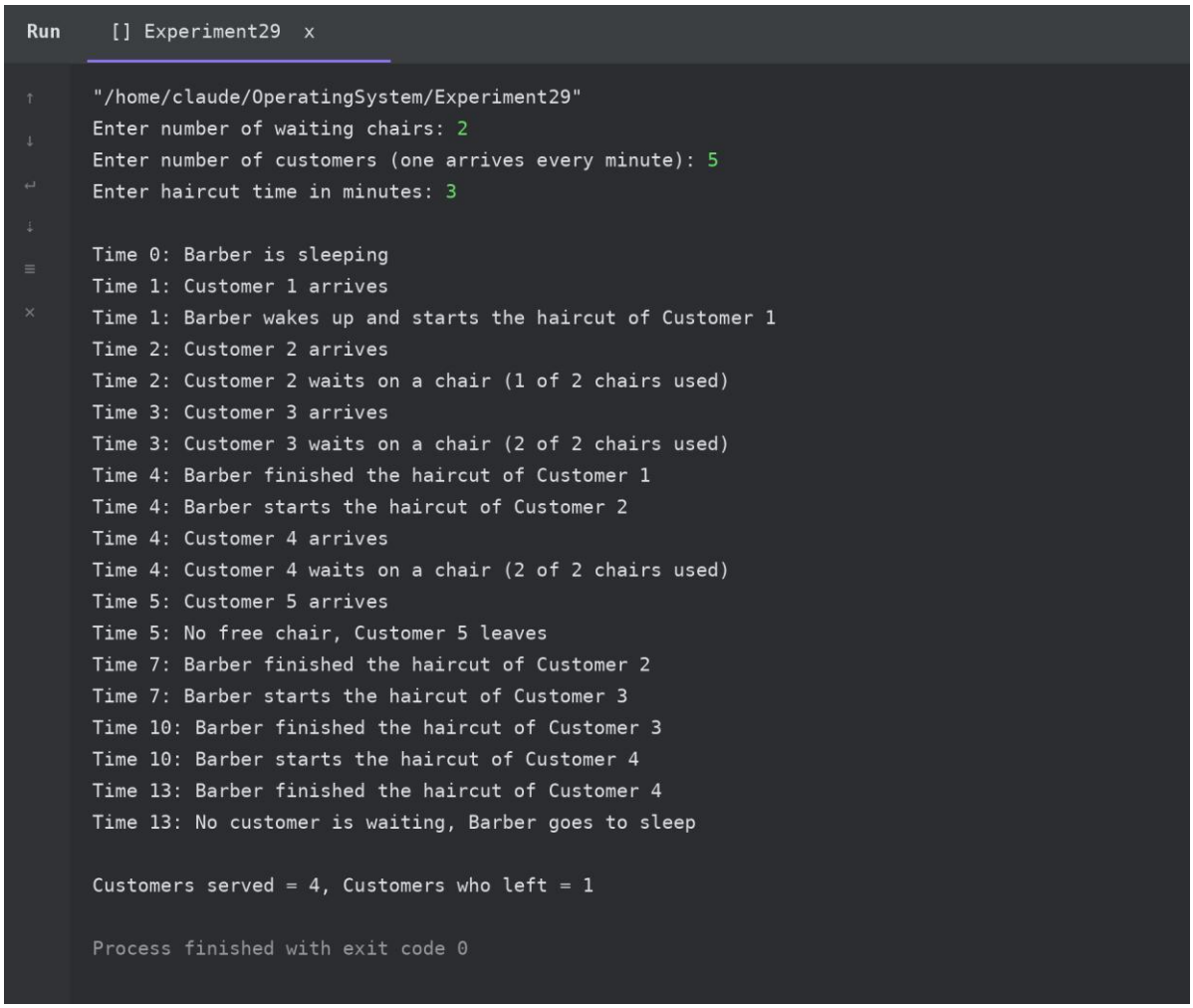
```
#include <stdio.h>

// Sleeping Barber problem: one barber, a few waiting chairs, customers arrive
every minute
int main() {
    int chairs, n, cut, t, waiting = 0, queue[50], head = 0, tail = 0;
    int busy = 0, finish = 0, cur = 0, served = 0, left = 0, fin;

    printf("Enter number of waiting chairs: ");
    scanf("%d", &chairs);
    printf("Enter number of customers (one arrives every minute): ");
    scanf("%d", &n);
    printf("Enter haircut time in minutes: ");
    scanf("%d", &cut);

    printf("\nTime 0: Barber is sleeping\n");
    for (t = 1; served + left < n; t++) {
        fin = 0;
        if (busy && t == finish) {
            printf("Time %d: Barber finished the haircut of Customer %d\n", t,
cur);
            busy = 0;
            served++;
            fin = 1;
        }
        if (!busy && head < tail) {                 // take the next waiting
customer
            cur = queue[head++];
            waiting--;
            busy = 1;
            finish = t + cut;
            printf("Time %d: Barber starts the haircut of Customer %d\n", t,
cur);
        } else if (fin) {
            printf("Time %d: No customer is waiting, Barber goes to sleep\n", t);
        }
        if (t <= n) {                               // customer number t arrives
now
            printf("Time %d: Customer %d arrives\n", t, t);
            if (!busy) {
                printf("Time %d: Barber wakes up and starts the haircut of
Customer %d\n", t, t);
                cur = t;
                busy = 1;
                finish = t + cut;
            } else if (waiting < chairs) {
                queue[tail++] = t;
                waiting++;
                printf("Time %d: Customer %d waits on a chair (%d of %d chairs
used)\n", t, t, waiting, chairs);
            } else {
                left++;
                printf("Time %d: No free chair, Customer %d leaves\n", t, t);
            }
        }
    }
    printf("\nCustomers served = %d, Customers who left = %d\n", served, left);
    return 0;
}
```

Output Screenshot:



Experiment 30

Write C programs to demonstrate the following thread related concepts.
(i) create (ii) join (iii) equal (iv) exit

Executable C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void *worker(void *arg) {
    int n = *(int *)arg;
    int *res = malloc(sizeof(int));
    usleep(n * 100000);                         // let main print first
    printf("Thread running: received value %d\n", n);
    *res = n * n;
    pthread_exit(res);                          // (iv) exit: terminate the thread
and return a value
}

int main() {
    pthread_t t1, t2;
    int a, b, *r1, *r2;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    // (i) create
    pthread_create(&t1, NULL, worker, &a);
    pthread_create(&t2, NULL, worker, &b);
    printf("create : thread t1 and thread t2 created\n");

    // (iii) equal
    printf("equal  : t1 and t1 are %s\n", pthread_equal(t1, t1) ? "equal" : "not
equal");
    printf("equal  : t1 and t2 are %s\n", pthread_equal(t1, t2) ? "equal" : "not
equal");
    printf("equal  : main thread and t1 are %s\n", pthread_equal(pthread_self(),
t1) ? "equal" : "not equal");

    // (ii) join
    pthread_join(t1, (void **)&r1);
    printf("join   : t1 finished, returned square = %d\n", *r1);
    pthread_join(t2, (void **)&r2);
    printf("join   : t2 finished, returned square = %d\n", *r2);

    free(r1);
    free(r2);
    return 0;
}
```

Output Screenshot:

